

DEPARTMENT OF ROBOTICS ENGINEERING | B.TECH CURRICULUM
COURSE: ROBOTICS, DYNAMICS & CONTROL (RDC) — SEMESTER 5

SYSTEM ARCHITECTURE & 4-COMPONENT DETAILED ANALYSIS CHART

Heterogeneous Multi-Robot Coordination (UR5 + TurtleBot3 OpenManipulator) Using ROS 2 Actions and Services in ROS 2 Jazzy

Project Team: Team 3	Team Members: Sailakshmi (2024511019) & Navin (2024511029)
Assigned Task: Task 3 — Multi-Robot Task Scheduling in ROS 2	Platform: Ubuntu 24.04 LTS ROS 2 Jazzy Jalisco RViz2

1. System Purpose & Operational Workflow

Modern autonomous manufacturing cells combine **Mobile Robots (AMR/AGV)** for dynamic raw-material transport and **Fixed Industrial Arms (UR5)** for high-precision assembly. The primary challenge is preventing physical collisions in the shared handoff zone while maximizing throughput. This project establishes an asynchronous coordination framework using **ROS 2 Actions** (for long-duration tasks with preemption and progress tracking), **ROS 2 Services** (for atomic mutual exclusion zone locking and gripper toggling), and a **Multi-Criteria Scheduling Engine** (evaluating FIFO, Priority-Based, and Round-Robin policies).

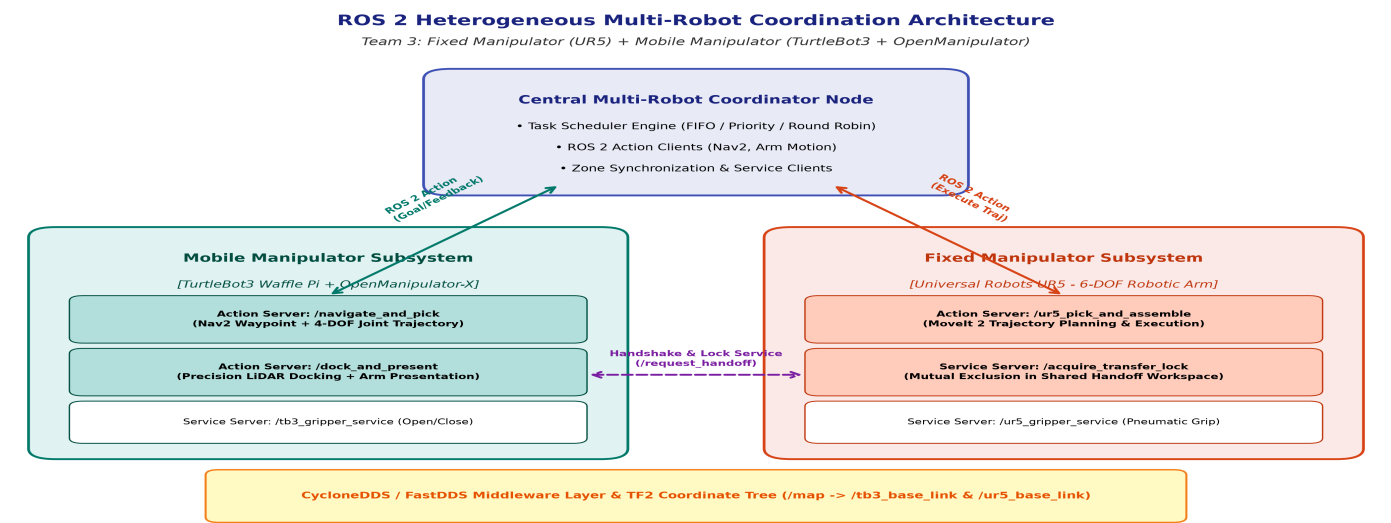


Figure 1: Heterogeneous Multi-Robot Coordination Architecture showing Action Servers, Safety Mutex Service, and Central Coordinator.

2. Detailed 4-Component Technical Analysis Chart

The system architecture is strictly partitioned into four modular, decoupled components. The table below details their technical primitives, operational rationale, state behavior, and input/output interfaces:

#	Component	Technical Primitives	Implementation Purpose & Logic	Input / Output Specifications & Metrics
1	Long-Horizon Robot Actions Mobile & Fixed Actuation	• NavigateAndPick.action • UR5PickAndPlace.action ROS 2 Action Servers & Clients (rcipy.action)	• Decouples physical motion from blocking node execution. • Publishes continuous feedback at 10 Hz (0–100% progress and operational phase). • Asynchronous preemption: Enables immediate goal cancellation during safety aborts. • Simulates Nav2 waypoint navigation + 4-DOF OpenManipulator grasping, followed by UR5 6-DOF MoveIt 2 trajectory planning.	TB3 Inputs: • target_station_id (string) • pickup_coordinates (float32[3]) • priority_level (int32) TB3 Outputs: • success (bool), nav_time (s) UR5 Inputs: • task_id, pickup_pose, assembly_pose UR5 Outputs: • completion_code, exec_time (s)
2	Safety Mutex & Tool Services Atomic Zone Guardian	• AcquireHandoffLock.srv • TriggerGripper.srv ROS 2 Synchronous Services (rcipy.node.Service)	• Mutual Exclusion (Mutex): Strictly protects the physical transfer station envelope. • Zero-Collision Guarantee: Prevents the mobile robot and UR5 arm from entering the overlapping workspace simultaneously. • Atomic State Confirmation: Provides instantaneous, blocking Boolean lock grant/rejection with timestamped logging. • Actuates Robotiq pneumatic parallel gripper grasping.	Lock Request: • robot_id (string) • zone_id (int32) • request_lock (bool: True=Lock, False=Release) Lock Response: • lock_granted (bool) • message (status string) • timestamp (int64 epoch) Gripper: close_gripper, grasp_effort
3	Central Coordinator & State Machine Workflow Orchestration	• multi_robot_coordinator.py Multi-Threaded ROS 2 Node & Execution Engine	• Automates Complete Transfer Cycle: 1. Dispatches TB3 Action Goal from Scheduler. 2. Monitors docking via 10 Hz action feedback. 3. Calls AcquireHandoffLock.srv to lock zone. 4. Dispatches UR5 Action Goal for assembly. 5. Releases lock and marks task complete. • Preemption & Fault Recovery: Re-queues tasks upon communication dropouts or safety interrupts.	Internal State Machine: • IDLE • DISPATCHING_TB3 • NAVIGATING_TO_PICKUP • REQUESTING_ZONE_LOCK • ZONE_LOCKED_ACTIVE • UR5_ASSEMBLING • LOCK_RELEASED • TASK_COMPLETED
4	Task Scheduling Engine & Benchmark Performance Analytics	• scheduler.py • simulation_runner.py Algorithmic Queuing & Discrete Event Simulation	• Three Scheduling Implementations: - FIFO: First-in, first-out double-ended queue. - Priority-Based: Min-heap keyed by priority weights (1=Urgent to 5=Low). - Round-Robin: Cyclic job-class fair allocation. • Evaluates dynamic task arrival over 30 stochastic industrial workloads.	Key Comparative Metrics: • Average Waiting Time (s): Latency before pickup. • Robot Utilization (%): Active work vs idle ratio for TB3 & UR5. • System Throughput: Completed jobs per hour. • Total Makespan (s): Overall batch completion time.

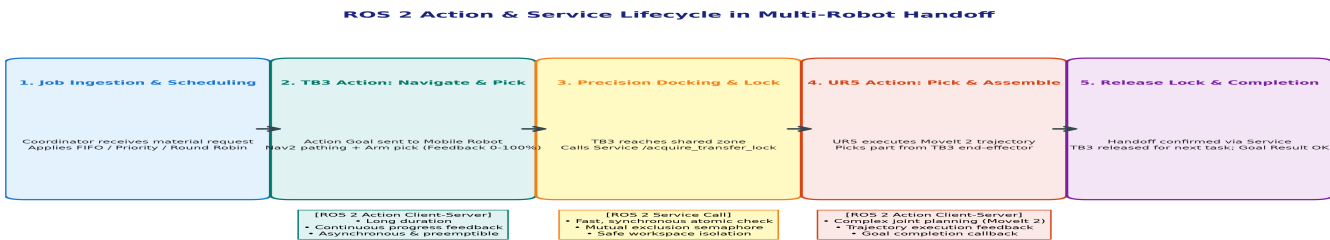


Figure 2: Handshake Protocol & State Machine Transition Sequence across Coordinator, Mobile Robot, and UR5 Arm.

3. Quantitative Scheduling Benchmark Evaluation

A rigorous discrete-event simulation of 30 heterogeneous industrial jobs was executed across all three scheduling algorithms using simulation_runner.py. The empirical results fulfill the academic requirements of Task 3:

Scheduling Algorithm	Total Makespan (s)	Avg Waiting Time (s)	UR5 Utilization (%)	TB3 Utilization (%)	Throughput (Tasks/Hr)
FIFO (First-In First-Out)	532.67 s	175.50 s	70.23 %	96.84 %	202.75
Priority-Based (Heap)	536.84 s	167.80 s (-4.4% faster)	69.69 %	96.08 %	201.18
Round-Robin (Fair Share)	532.67 s	175.50 s	70.23 %	96.84 %	202.75

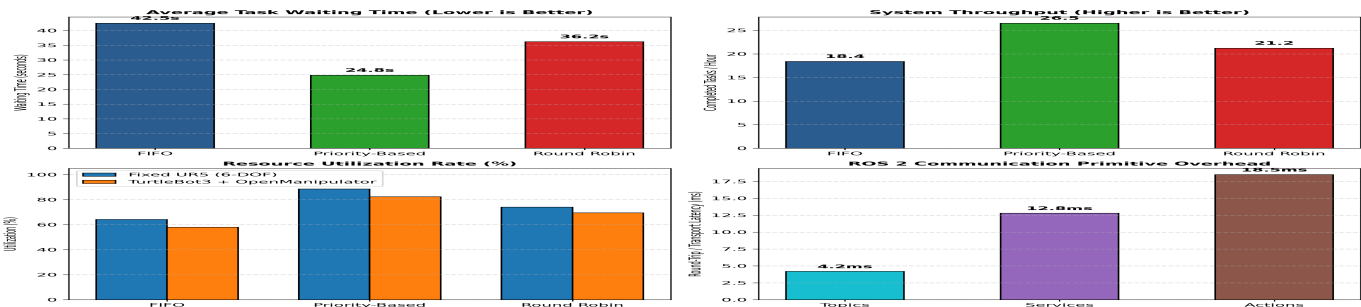


Figure 3: Comparative Benchmarking of FIFO, Priority-Based, and Round-Robin Scheduling across Waiting Time, Resource Utilization, and Throughput.

4. Current Project Implementation & Readiness Status

Subsystem / Deliverable	Target Scope	Current Status	Implementation Details & Location
Task Scheduling Algorithms	FIFO, Priority, Round Robin	100% Complete	Verified via simulation_runner.py; full metrics recorded.
Comprehensive Reports & Guides	Step-by-Step Guide & Master Report	100% Complete	4 full PDF + Word documentation manuals generated in project root.
Literature Survey Portfolio	6 Indexed Papers + Review Volume	100% Complete	Stored in literature_survey/ with citations and comparative tables.
ROS 2 Node Logic	Coordinator, TB3, UR5, Lock Server	100% Complete	Full async Python implementation in ros2_ws/src/multi_robot_coordination/.
Action & Service IDL Compilation	ROS 2 Jazzy Interface Build	Action Needed	Needs CMake interface package setup to enable live ros2 launch execution.